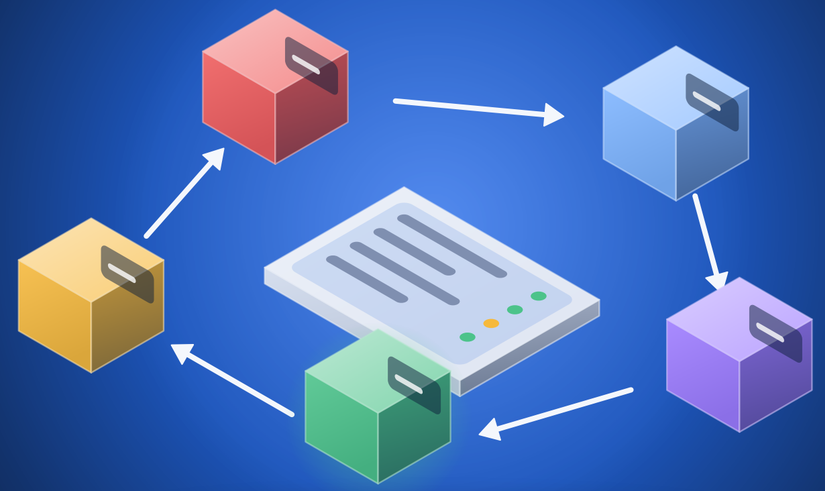
Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · WEEK 3

Processes

What a process is, how the operating system creates, schedules and ends processes, and how processes communicate.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Identify the parts of a process and how the operating system represents them.
2. Describe how processes are created and terminated, using the system calls involved.
3. Compare interprocess communication by shared memory and by message passing.
4. Write programs that use pipes and POSIX shared memory.
5. Describe client–server communication with sockets and remote procedure calls.

THIS WEEK

Topic 3

The process is the unit of work in a modern system. Scheduling, synchronization and memory management all act on processes.

DISCUSSION

Why does one browser show dozens of processes?

Open the task manager while a browser runs. One program, many processes. By the end of this session you can explain why.

PART 1

The process concept

A program in execution, and everything the system must remember about it.

Week 3

Program and process

A program is a *passive* file on disk. A process is that program *in execution*.

Becoming a process

A program becomes a process when its executable file is loaded into memory, by a double-click or a command.

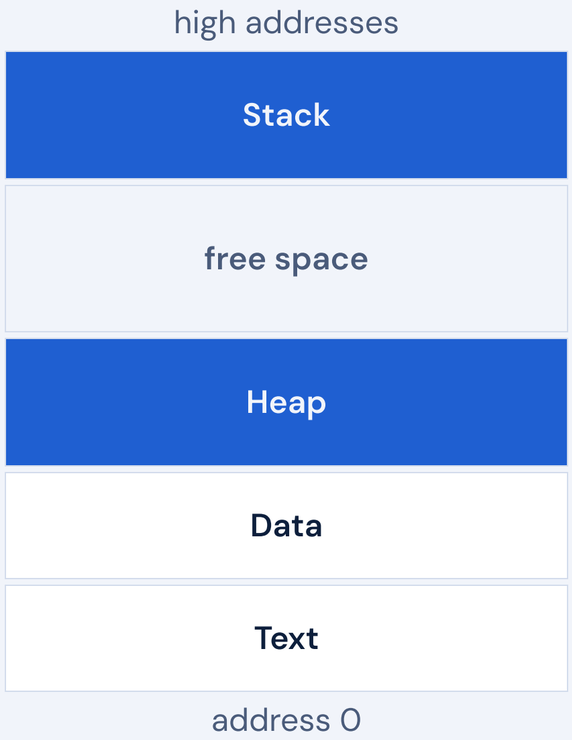
One program, many processes

Several users can run the same program. Each run is a separate process with its own state.

Sequential execution

A process executes its instructions one after another. It needs CPU time, memory, files and I/O devices.

The memory layout of a process



Text and data

Text is the executable code. Data holds the global variables. Both have a fixed size.

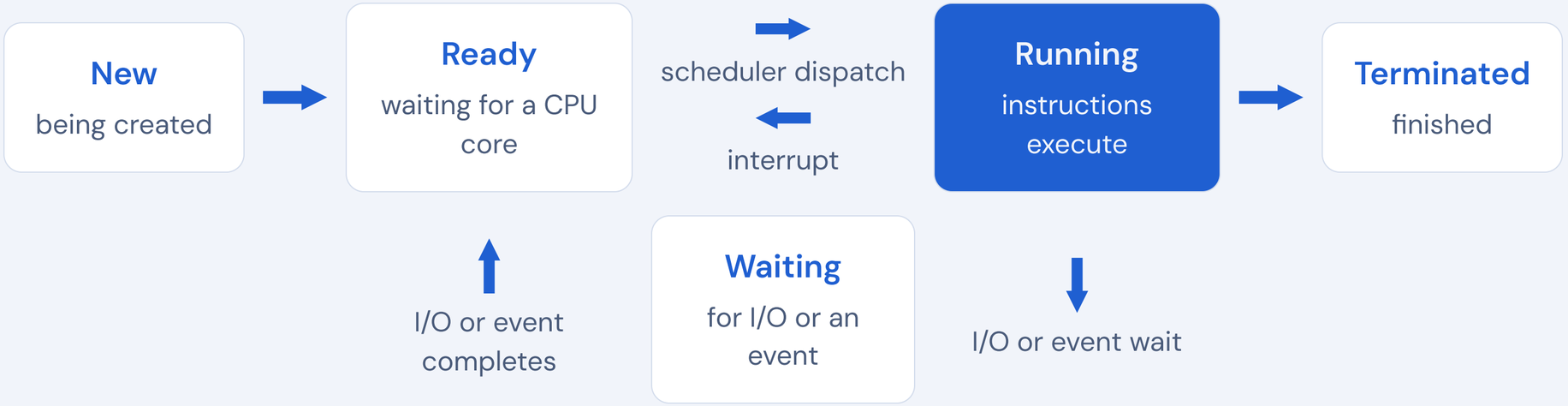
Heap

Memory allocated while the program runs, with malloc() for example. It grows upward.

Stack

Temporary data of each function call: parameters, return address and local variables. It grows downward.

Process states



Only one process can be running on a core at any instant. Many may be ready or waiting.

The process control block

- process state
- process number
- **program counter**
- CPU registers
- scheduling information
- memory limits
- list of open files

The operating system keeps one PCB for each process.

It holds everything needed to stop a process now and continue it later as if nothing had happened.

The program counter gives the address of the next instruction. The registers hold the values the process was working with.

In Linux the PCB is the C structure task_struct, and all the active processes are on a doubly linked list of these.

A first look at threads

ONE THREAD

So far

A process has a single thread of execution: one program counter, one task at a time.

MANY THREADS

With threads

A process has several program counters, so several parts of it can run at the same time. The PCB is extended with information for each thread.

Threads are the subject of Week 4.

PART 2

Process scheduling

Keep the CPU busy, and switch among processes fast enough for users to interact with each one.

Scheduling queues



Ready queue

All processes that are in main memory, ready and waiting to execute on a CPU core.



Wait queues

Processes waiting for an event, such as the completion of I/O. There is one queue for each kind of event.



Process scheduler

It selects one process from the ready queue for the next execution on a core. Processes move between the queues all their lives.

The path of a process through the queues



I/O request

It waits in an I/O wait queue.

Time slice expired

It goes straight back to the ready queue.

Creates a child

It may wait for the child to terminate.

Interrupt

It is put back in the ready queue.

The context switch



Context

The state of a process, as recorded in its PCB.

Pure overhead

During a switch the system does no useful work. A complex operating system and a large PCB make it longer.

Hardware matters

Some processors keep several sets of registers, so several contexts can be loaded at once.

PART 3

Operations on processes

Creating a process and ending one.

Week 3

Process creation

A *parent* process creates *children*, which create others: a tree of processes.

Identity

Every process is identified and managed by a process identifier, the pid.

Sharing resources

Parent and child may share all resources, share some of them, or share none.

Execution

The parent may continue to run at the same time as its children, or wait until they terminate.

A tree of processes in Linux



systemd is the first process, started at boot, and the ancestor of all the others. The command pstree shows this tree on your own machine.

Creating a process in UNIX

```
pid_t pid = fork();

if (pid < 0) { /* error */
    perror("fork");
} else if (pid == 0) { /* child */
    execlp("ls", "ls", "-l", NULL);
} else { /* parent */
    wait(NULL);
    printf("child finished\n");
}
```

fork()

Creates a new process: a copy of the parent's address space. It returns 0 in the child and the child's pid in the parent.

exec()

Replaces the process's memory with a new program.

wait()

The parent waits for the child to terminate.

Creating a process in Windows

CREATE AND LOAD

CreateProcess()

One call does the work of fork() and exec() together: it creates the child and loads the named program into it. It takes at least ten parameters.

WAIT

WaitForSingleObject()

The parent waits for the child to finish, as wait() does in UNIX. It then closes its handles to the child.

Two designs for the same job. UNIX separates creating a process from loading a program. Windows combines them.

Process termination

exit()

A process finishes its last statement and asks the operating system to delete it. It returns a status value to its parent, and its resources are released.

abort()

A parent may end a child: the child used too many resources, its task is no longer needed, or the parent itself is exiting.

Cascading termination

Some systems do not let a child live when its parent has ended. All children, and their children, are terminated.

Zombies and orphans

Zombie

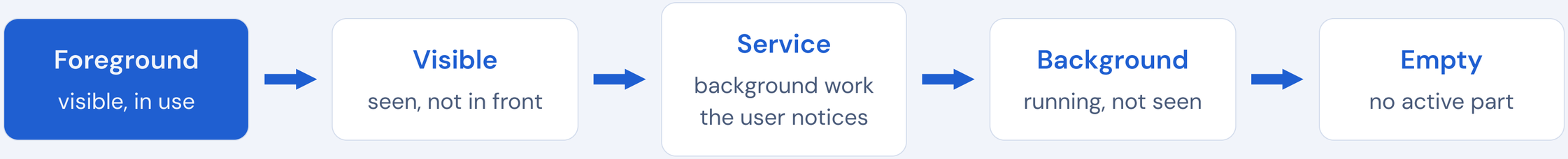
A process that has terminated, but whose parent has not yet called wait(). Its entry stays in the process table until the status is collected.

Orphan

A process whose parent terminated without calling wait(). In Linux it is adopted by systemd, which collects its status.

wait() returns the pid of the terminated child and gives the parent its exit status.

Which process to end first: Android



A phone has little memory. When it must reclaim some, Android ends processes from the least important first: from the right of this line to the left.

A multiprocess browser



Browser process

Manages the user interface, the disk and the network. There is one.



Renderer processes

Each renders a web page: HTML, images and JavaScript. Usually one per tab, each in a sandbox with limited access.



Plug-in processes

One for each type of plug-in in use.

This answers the opening question. If one site crashes, only its renderer process fails. A single-process browser would lose every tab.

PART 4

Interprocess communication

Cooperating processes need a way to exchange data.

Independent and cooperating processes

ALONE

Independent

It does not share data with any other process, so it cannot affect or be affected by them.

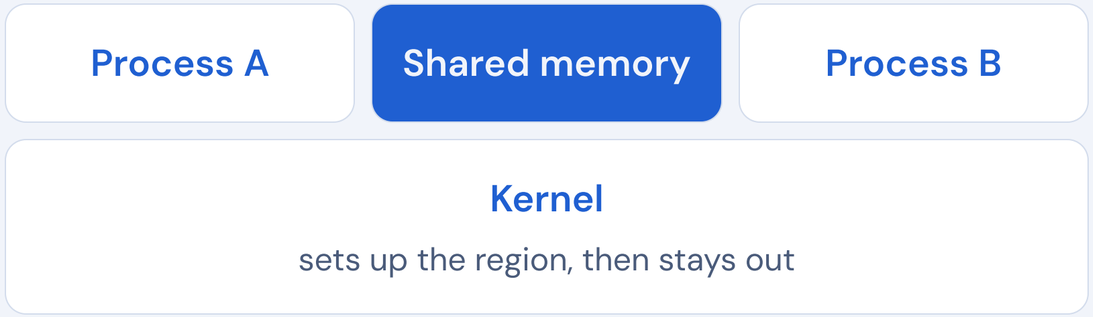
TOGETHER

Cooperating

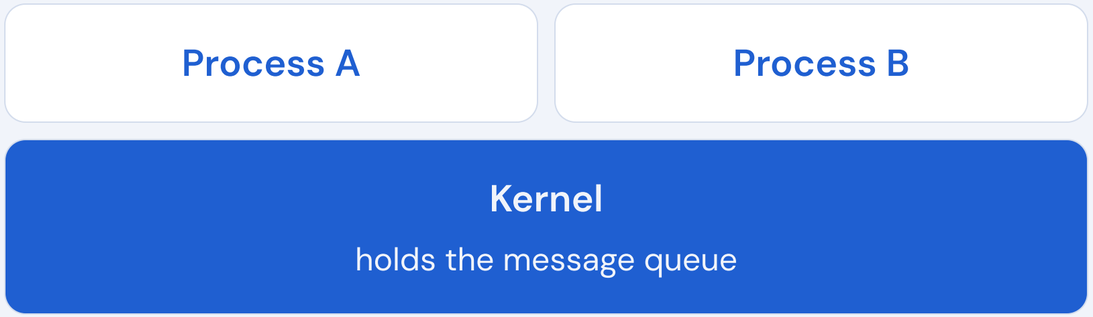
It can affect or be affected by other processes. Reasons to cooperate: sharing information, speeding up a computation, and building a system in modules.

Cooperating processes need interprocess communication, IPC. There are two models: shared memory and message passing.

Two models of IPC

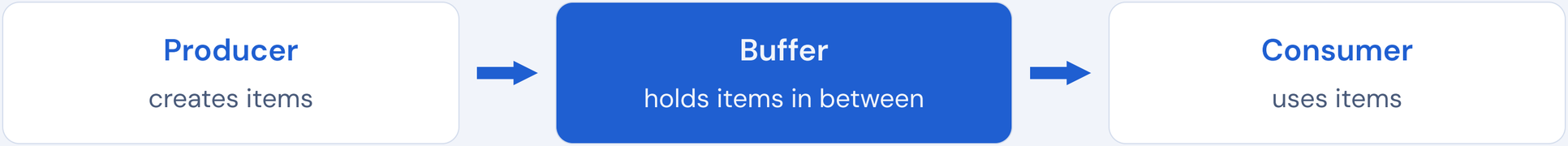


Shared memory: both processes read and write one region. Fast, since the kernel is needed only to set it up.



Message passing: every message goes through the kernel. Slower, but simpler for small amounts of data and across machines.

The producer–consumer problem



Unbounded buffer

No practical limit on its size. The producer never waits. The consumer waits if the buffer is empty.

Bounded buffer

A fixed size. The consumer waits if the buffer is empty, and the producer waits if it is full.

This small problem returns throughout the course as the standard example of cooperating processes.

A bounded buffer in shared memory

```
#define N 8
item buffer[N];
int in = 0, out = 0;

/* producer */
while (((in + 1) % N) == out)
    ; /* full: wait */
buffer[in] = next;
in = (in + 1) % N;
```



A circular array. **out** is the first full position, here 2. **in** is the next free position, here 5.

Empty when in equals out. Full when the position after in equals out.

So this version can use only N − 1 of its N positions.

A counter, and a race condition

To use all N positions, we add a shared variable **count**. The producer does count++ and the consumer does count−−.

Each of these is three machine steps: load into a register, change the register, store back.

If the two processes run those steps interleaved, the final value depends on the order. That is a race condition.

Step	Who	Action	Value
1	Producer	r1 = count	r1 = 4
2	Producer	r1 = r1 + 1	r1 = 5
3	Consumer	r2 = count	r2 = 4
4	Consumer	r2 = r2 - 1	r2 = 3
5	Producer	count = r1	count = 5
6	Consumer	count = r2	count = 3

One item was produced and one consumed, so count should still be 4. It is 3. Week 6 deals with this problem.

Message passing

Processes communicate with two operations: *send* and *receive*.

A communication link

Two processes that want to communicate must first establish a link between them, then exchange messages over it.

Physical

The link may be shared memory, a hardware bus or a network.

Logical

Direct or indirect. Synchronous or asynchronous. With automatic or explicit buffering.

Direct and indirect communication

NAME THE PROCESS

Direct

Each process names the other: send(P, message) and receive(Q, message). A link is created automatically, between exactly one pair of processes.

NAME THE MAILBOX

Indirect

Messages go to and come from a mailbox, also called a port. Each mailbox has a unique id. A link exists only if the processes share a mailbox, and it may join many processes.

If three processes share a mailbox and one message arrives, who gets it? The system must decide: allow two processes per link, allow one receiver at a time, or choose a receiver itself.

Blocking and buffering

	Blocking	Non-blocking
Send	The sender waits until the message is received	The sender sends and continues
Receive	The receiver waits until a message is available	The receiver gets a message or a null value

Zero capacity

No messages wait on the link. The sender must wait for the receiver: a rendezvous.

Bounded capacity

A finite number of messages. The sender waits only if the link is full.

Unbounded capacity

Any number of messages. The sender never waits.

PART 5

IPC in real systems

Shared memory, messages and pipes as operating systems provide them.

POSIX shared memory

```
int fd = shm_open("/notes", O_CREAT | O_RDWR, 0666);
ftruncate(fd, 4096);

char *p = mmap(0, 4096, PROT_READ | PROT_WRITE,
               MAP_SHARED, fd, 0);

sprintf(p, "hello from the producer");
```

Three steps.

shm_open() creates a shared-memory object, or opens an existing one, by name.

ftruncate() sets its size.

mmap() maps the object into the address space. After that, the process uses it through an ordinary pointer.

The consumer opens the same name, maps it, and reads what the producer wrote.

Message passing in Mach and Windows

Mach

Everything is a message, even a system call. Messages go to ports, which are mailboxes. Each task starts with two ports: one for the kernel and one for notifications. The kernel of macOS and iOS uses this.

Windows

Advanced local procedure calls, ALPC, between processes on the same machine. A client opens a handle to the server's connection port. The server creates a private channel, with one port for each direction.

Pipes

UNNAMED

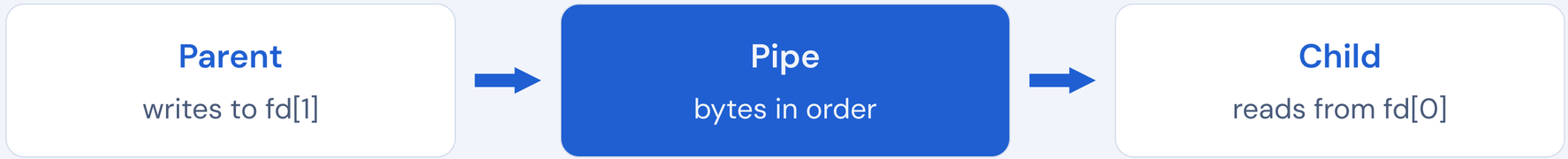
Ordinary pipes

One direction only: a write end and a read end. They need a parent–child relationship, so they are used between a process and the child it created. Windows calls them anonymous pipes.

NAMED

Named pipes

Two directions. No parent–child relationship is needed, several processes can use the same pipe, and the pipe remains after the processes end.



PART 6

Communication in client–server systems

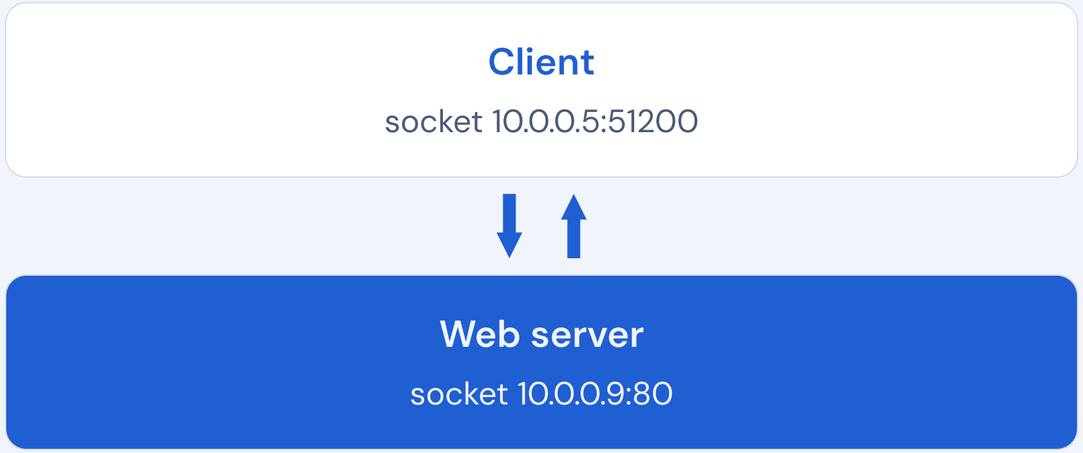
When the two processes are on different machines.

Sockets

A socket is one *endpoint* of a communication.

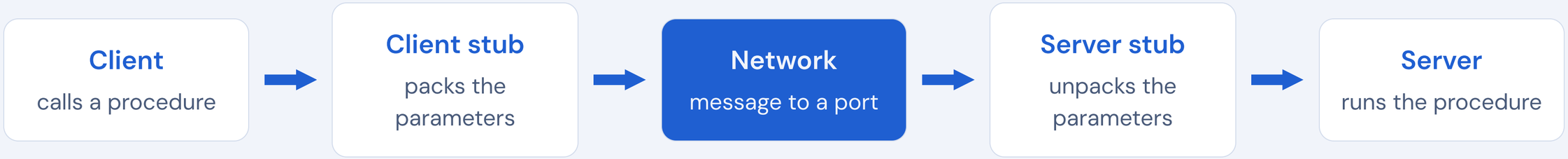
It is identified by an IP address together with a port number. A connection is a pair of sockets, one on each side.

Ports below 1024 are well known and used by standard services. 127.0.0.1 is the loopback address: the machine itself.



Three kinds: connection-oriented (TCP), connectionless (UDP), and multicast, which sends to several recipients.

Remote procedure calls



Stubs

The client calls a local stub as if it were the real procedure. The stub hides the communication.

Marshalling

Packing the parameters in a form both machines understand, since they may store data differently.

Failures

A network can lose or repeat a message. The system aims for each call to be executed exactly once, or at most once.

Key takeaways

01

A process is a program in execution

With text, data, heap and stack, and one PCB that records its state.

02

Processes move between queues

Ready, running, waiting. A context switch saves one process and loads the next.

03

fork, exec, wait, exit

The four calls behind creating and ending processes in UNIX.

04

Two models of IPC

Shared memory is fast. Message passing is simpler and works across machines.

Next session

COMING UP

Threads and concurrency

Several threads inside one process, multicore programming, and thread libraries.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 3.